Launcher ABI (bootstrap → runtime)

TBK SPEC 06:2026, Version 1.0

The Tebako Project

Unrestricted
November 01, 2026
TBK SPEC 06:2026, Version 1.0
Tebako
© 2026 The Tebako Project. All rights reserved

Ribose

Contents

Foreword 4

1. The handoff 4

2. Image-era addition (additive, ABI stays 1) 5

3. Loader behavior contract (tebako-bootstrap) 5

4. Exit codes (named, stable) 6

5. Progress UX (locked 2026-07-26) 7

5a. The fetch-plan rendering (tebako-term v2, locked 2026-09-18) 8

6. Bootstrap↔runtime contract negotiation (roadmap 45) 9

© 2026 The Tebako Project

All rights reserved. Unless otherwise specified, no part of this publication may be reproduced or utilized otherwise in any form or by any means, electronic or mechanical, including photocopying, or posting on the internet or an intranet, without prior written permission. Permission can be requested from the address below.

The Tebako Project

Suite 1, 8/F, New Henry House
10 Ice House Street
Central
Hong Kong

copyright@ribose.com
www.ribose.com

© 2026 The Tebako Project

Foreword

Normative specification of the handoff contract. Version: 1. Status: SHIPPED (macOS/Linux; Windows exec/lock port PARTIAL — roadmap 02).

1. The handoff

FIGURE 1

```
<runtime> --tebako-image <self>:<slot>:<mount> ...
          --tebako-entry <argv0> <user args...>
```

- One `--tebako-image` triple per payload slot to mount: `<self>` = the package's own path, `<slot>` = slot index, `<mount>` = the slot's mount point.
- Slots whose role is runtime are **never** handed over as mounts.
- `--tebako-entry` separates loader-consumed args from user args; `<argv0>` is the entrypoint name inside the mounted tree.
- `--tebako-extract` is a runtime-side option riding the user-arg passthrough (the loader never interprets it).

© 2026 The Tebako Project

2. Image-era addition (additive, ABI stays 1)

When the runtime_ref carries `;image` (spec 05 §1), the loader exports:

FIGURE 2

```
TEBAKO_RUNTIME_IMAGE=<absolute path of the cached .tfs>
```

The runtime driver prefers the env image over any embedded image (one-file driver patch, `docs/tebako-main.cpp.30b.patch`); v1 runtimes ignore the env and use their embedded image — graceful degradation, no republish of v1-era runtimes needed.

3. Loader behavior contract (tebako-bootstrap)

1. Read own trailer (spec 02; absent → classic-bundle error path).
2. Require `launcher_abi == 1` (else exit 66).

2a `--tebako-install` verb: after the chain + ABI gates — the package's `TPKG_FLAG_NO_INSTALL` refuses (exit 76); any other package answers with the named guidance to `tebako install <path>` (exit 76 — the manifest read needs the TFS engine the size-capped bootstrap deliberately does not carry). A run is a run: the verb is the ONLY way slices reach the store.

3. Trust handling, by build and trailer flag:
   - `TPKG_FLAG_SIGNED_V2`, verification ENABLED (`openpgp-verify` feature): verify the OpenPGP signature against the trusted keyring and each slot's SHA-256 — always strict (spec 09).
   - `TPKG_FLAG_SIGNED_V2`, verification DISABLED (unverified-first, the shipped default until roadmap 72's crypto toolkit): loud UNVERIFIED warning + audit journal, then enforce each slot's SHA-256 as integrity-vs-corruption (the anchor is unverified — documented, spec 09 §7). `TEBAKO_REQUIRE_SIGNED=1` here fails CLOSED with exit 71 naming the missing capability ("built without OpenPGP verification") — a strict-mode request is never silently downgraded to unverified.

© 2026 The Tebako Project

- Unsigned (v1): loud warning + audit journal (or exit 71 under `TEBAKO_REQUIRE_SIGNED=1`).

4. Resolve the runtime per spec 05 §5 — negotiating the contract version (§6) fail-closed before any checksum acceptance.
5. Image-era: ensure `<asset>.tfs` + trust markers in the cache entry (fetch + verify on miss — authenticity per spec 09 §4's runtime-fetch point first, then sha256 integrity as today), install read-only.
6. Exec the handoff. Never returns on success.

4. Exit codes (named, stable)

TABLE 1

code	name	meaning
65	`EX_TEBAKO_MANIFEST`	trailer missing/corrupt/invalid
66	`EX_TEBAKO_ABI`	launcher_abi mismatch
67	`EX_TEBAKO_RUNTIME_REF`	unparseable/unsupported runtime_ref
68	`EX_TEBAKO_OVERLAY`	overlay/decrypt binding failure: unbound retained store, missing or non-opening key material, unwritable store, orphan binding, malformed `TEBAKO_OVERLAYS` / `TEBAKO_DECRYPT` (spec 24 §7; code constant `tpkg::EX_TEBAKO_OVERLAY`)
69	`EX_TEBAKO_UNAVAILABLE`	runtime unresolvable (offline miss, download failure)
70	`EX_TEBAKO_SHA`	sha256 mismatch (runtime or image)
71	`EX_TEBAKO_SIGNATURE`	invalid signature; or unsigned under `TEBAKO_REQUIRE_SIGNED=1`
72	`EX_TEBAKO_TRUST`	signer key not in the trusted keyring
73	`EX_TEBAKO_JAIL`	jail policy could not be applied (malformed `TEBAKO_JAIL`; fail-closed — spec 08)
74	`EX_TEBAKO_IO`	filesystem/lock/install failure
75	`EX_TEBAKO_CONTRACT`	runtime declares an unsupported `contract_version` (§6)
76	`EX_TEBAKO_INSTALL`	`--tebako-install` refused (`TPKG_FLAG_NO_INSTALL`) or needs the CLI

© 2026 The Tebako Project

code	name	meaning
77	EX_TEBAKO_CONTRACT_ERA	the runtime declares a contract ERA newer than the loader speaks (§6; code constant tpkg::EX_TEBAKO_CONTRACT_ERA)
78	EX_TEBAKO_LAYOUT	the TEBAKO_MOUNT_ROOT override is ungranted by the env image's mount_root_override layout grant (spec 17 §1; code constant tebako-driver's EX_TEBAKO_LAYOUT)
79	EX_TEBAKO_CHECK	a payload check FAILed — the tebako check aggregate (spec 26 §2; code constant tpkg::EX_TEBAKO_CHECK)
80	EX_TEBAKO_LEDGER	a ledger finding — tebako audit --verify met a chain mismatch or an invalid seal signature, the seal file holds an unparseable record, or --strict met unparseable journal lines (spec 40 §6; code constant tpkg::EX_TEBAKO_LEDGER)
81	EX_TEBAKO_SOLVE	a tree-solve finding — a named conflict, lockfile drift under --locked, an open runtime seam, or a malformed requirement set (spec 42 §5; code constant tebako_cli::solve::EX_TEBAKO_SOLVE)

stderr body: `tebako-bootstrap: <message>\n` — message bodies match the C++ reference bootstrap 1:1 (golden parity).

5. Progress UX (locked 2026-07-26)

When the loader fetches a runtime or image, the user SEES the work and the benefit. Rules:

- **TTY-only:** full progress rendering iff stderr is a TTY and `TERM != dumb`; otherwise exactly two single lines (start + done), CI/ log-safe. Opt-outs: `NO_COLOR`, `TEBAKO_NO_PROGRESS=1`, and the package's baked `TPKG_FLAG_QUIET_NOTICES` (bit 3 — spec 23 §14).
- **Phases, one line each:** `resolving <runtime_ref>` → `downloading <asset> (<size>)` with the live bar → `verifying sha256` → `installing (locked)` → done.
- **The bar** (hand-rolled ANSI, no deps — the size gate forbids indicatif-class crates): `\r[=====> ] 62% 14.2/23.0 MB 3.1 MB/s` throttled to ≤ 10 redraws/s; unknown content-length → spinner frames + byte count.
- **The benefit is stated:** on completion — `installed ruby-3.4.2-0.15.9-linux-gnu-x86_64 (23.0 MB) — cached at ~/.tebako/runtimes/… and shared by every tebako app on this machine`. A cache HIT prints one quiet line: `runtime ruby-3.4.2 (cached)`. `TEBAKO_NO_PROGRESS=1` silences these informational lines entirely (the cache-hit, `installed …`, and `downloading …` lines — progress is not results; tebako#400); the mode selection rule above still applies. `TPKG_FLAG_QUIET_NOTICES` (bit 3 — the developer's press-time declaration via the spec 23 §14 registry) applies the same silence to every run of the package, on every machine.

© 2026 The Tebako Project

- Progress output goes to **stderr**, never stdout (stdout belongs to the payload).
- Implementation: a tiny `tebako-term` micro-crate (TTY detect, bar, spinner, phase lines) consumed by tebako-bootstrap; tebako-shim and tebako-cli reuse it (no third copy). tebako-http gains an `on_progress(bytes_so_far, content_length)` callback hooking the stream — the bar is transport-accurate, not estimated.

5a. The fetch-plan rendering (tebako-term v2, locked 2026-09-18)

The spec 05 §6 fetch pipeline downloads a plan's artifacts CONCURRENTLY, and the user sees that concurrency — the docker-pull look. tebako-term v2 adds the plan surface (`ProgressSet`); the single-artifact contract above is its one-slot case.

- **The plan header** (both modes, one line, printed before the first artifact starts): `fetching <what>: <N> artifacts, <total> total` where `<what>` names the resolution subject (e.g. `runtime ruby 3.3.12`) and `<total>` sums the known size hints (`unknown` when none are known). A cache hit for the whole plan prints no header — the hit lines stand alone as before.
- **One live line per concurrent worker** (TTY mode): the set renders as a fixed block of N slot lines at the bottom of the scrollback, repainted in place (`\x1b[<N>A` up + erase-line per slot), throttled to ≤ 10 redraws/s for the WHOLE block (not per slot). Each slot runs the phase sequence **download bar → verifying (spinner) → done**; the done state keeps its line in the block until the plan completes (`✓ <asset> (<size>)`), a failed slot renders `# <asset>` and the plan's named error follows on a fresh line.
- **The bar grows up:** fractional unicode blocks (▏▎▍▌▋▊▉ — eighth-cell resolution), an EMA-smoothed rate (`3.1 MB/s`, α = 0.3 per redraw) plus an ETA (`eta 12s`, shown only once the EMA has converged past its warmup), the byte pair in the total's unit. Unknown content-length → a braille spinner (`########⠋#`)
  - byte count. Colors: cyan for an active slot, green for done, red

    for failed — decorative only, and absent whenever the mode rule above selects plain (NO_COLOR included).
- **Plain mode grammar (pipe/CI/opt-out — byte-stable, testable):** exactly the plan header, then per artifact the pair `downloading <asset> (<size>)` / `installed <asset> (<size>)` (or `downloading <asset>` when the length is unknown), then the summary line `fetched <what>: <N> artifacts, <total> in <secs>s`. No control sequences, ever. `TEBAKO_NO_PROGRESS` quiets the whole grammar (the header included) — progress is informational, never results; failures are named errors on stderr regardless of the gate.
- **The wiring rule (locked):** every network artifact transfer — runtime pair downloads, payload slice fetches, registry index refreshes, trust-anchor key retrievals — renders through tebako-term; **no caller prints its own download progress**. A registry YAML fetch (small, unknown length) renders as a one-line spinner phase in TTY mode and a single `fetching registry <ref>` start line in plain mode (its done line is the resolution's own output). Callers that today emit nothing (the shim's dispatch-time

© 2026 The Tebako Project

fetch, the CLI's press resolver) gain the same surface — the download moment is one UX everywhere.

6. Bootstrap↔runtime contract negotiation (roadmap 45)

The launcher ABI (§1–§2) is the wire format; the **contract version** is its semantics version, declared by the runtime and enforced by the bootstrap. It exists so a future handoff change (env names, argv shape, image-handoff semantics) can be introduced without old bootstraps silently mis-executing new runtimes.

TABLE 2

contract	semantics
1	this document, §1–§5: `--tebako-image`/`--tebako-entry` argv, `TEBAKO_RUNTIME_IMAGE` env handoff, trailer/ABI gating as in §3
2	spec 17: image-path triples, bare-file slot tokens (`0` ≡ `-`), env-image-first multi-mount, direct `--tebako-entry` execution (no `/local/stub.rb` convention)

Bump rule. Any change to env/argv/handoff semantics — a renamed or repurposed variable, a new loader-consumed flag, a change in image precedence — increments the contract version by exactly 1. Additive payload-side changes that v1 runtimes may ignore (like §2's env image for v1 runtimes) do NOT bump the contract.

Declaration. The runtime release manifest's per-package entry carries the contract SET — `"contract_era"`, `"contract_version"`, `"mount_root"` — (spec 05 §5's manifest.json; the set is additive — older consumers ignore it). The runtime driver has the same values compiled in (`TEBAKO_CONTRACT_VERSION` and its siblings); the runtime repo's CI fails any release where the manifest and the driver disagree.

Negotiation rule (bootstrap, fail-closed — spec 18's era vocabulary). During runtime/image resolution, before any checksum acceptance, the loader reads the entry's contract set (`crates/tebako-resolve/src/contract.rs` is the implementation of record; the current spoken values are era 2 and contract 2 — the spec-17 argv/env grammar):

1. The manifest carries NO entry for the asset → not a contract question (the checksum path names that failure) → proceed.
2. The entry's contract set is present and matches the loader's spoken era + contract → proceed (the declared, acceptable set).

© 2026 The Tebako Project

3. Any of the three fields absent, unparseable, zero, or empty → a pre-era entry — a NAMED refusal (`ContractError::PreEra`, exit 75) listing the missing fields, never a proceed.
4. A declared era NEWER than the loader's → refused (`ContractError::EraTooNew`, exit 77 — `EX_TEBAKO_CONTRACT_ERA`) naming both generations and the remedy; a contract NEWER or OLDER than the spoken one → refused (`ContractTooNew`/`ContractTooOld`, exit 75) naming both.

A negotiation failure is never a crash and never a silent accept: it is a clean, explained refusal.

© 2026 The Tebako Project